

Reineke-RAG

Technisches Handbuch

Reineke-RAG — 2026

Reineke-RAG — Technisches Handbuch

Für Administratoren, DevOps, Architektinnen und Implementierer. Deckt ab, was Sie zum Deployen, Betreiben, Erweitern und zur Fehlersuche auf einem Single-Host-Docker-Compose-Deployment wissen müssen.

Vorher lesen: [TECH_DESCRIPTION_DE.md](#) für die konzeptionelle Übersicht. Dieses Handbuch setzt voraus, dass Sie wissen, was Reineke-RAG ist; es dokumentiert, *wie es betrieben wird*.

Autoritative Quellen im Repository: [docs/02_ARCHITECTURE.md](#), [docs/04_OPERATIONS.md](#), [docs/adr/](#). Dieses Handbuch ist die Arbeitsfassung — bei Abweichung gilt das `docs/ -` Verzeichnis, und diese Datei wird nachgezogen.

Inhalt

1. [Zielgruppe und Voraussetzungen](#)
2. [System-Überblick für Betreiber](#)
3. [Host-Vorbereitung](#)
4. [Installation \(online\)](#)
5. [Installation \(air-gapped\)](#)
6. [Tagesbetrieb — das rag-admin-CLI](#)
7. [Benutzer- und Gruppenverwaltung](#)
8. [Ordner und ACLs](#)
9. [Dokument-Ingestion und Lebenszyklus](#)
10. [Retrieval-Verhalten und Feintuning](#)
11. [Modell-Management](#)
12. [Observability](#)
13. [Backup und Restore](#)
14. [Security-Operations](#)
15. [Upgrades](#)
16. [Performance-Tuning](#)
17. [Fehlersuche](#)
18. [Das System erweitern](#)
19. [Anhänge](#)

1. Zielgruppe und Voraussetzungen

Dieses Handbuch setzt voraus, dass Sie sicher umgehen können mit:

- Docker Compose und Container-Lebenszyklus (`up` , `logs` , `exec` , `restart`).
- Einer POSIX-Shell und grundlegender Linux-/macOS-Systemadministration.

- OIDC- und JWT-Konzepten auf Anwender-Ebene (Issuer, Client, Scope, Groups-Claim).
- Basis-SQL und dem Konzept von Postgres-Rollen.
- Container-Logs lesen und aus einem Mischfehler den verursachenden Dienst identifizieren.

Sie brauchen **keine** ML-Engineer-Kenntnisse. Die Modell-Routing-Konfiguration ist deklarativ; die Retrieval-Pipeline ist nichts, was Sie in Produktion editieren — Sie drehen Stellschrauben (k-Werte, Chunk-Größe, Anzahl Rerank-Kandidaten).

Minimalwissen zum unbegleiteten Betrieb

Vor dem produktiven Betrieb sollten Sie:

- Erklären können, was Authentik tut und wie die Gruppenzugehörigkeit eines Benutzers im JWT eines Dienstes landet.
- Einen Langfuse-Trace lesen und auf den Reranker-Span zeigen können.
- `rag-admin backup run` ausführen und in ein Wegwerfverzeichnis zurückspielen können.

Fühlt sich eines dieser Themen unvertraut an, arbeiten Sie §13 (Backup und Restore) und §12 (Observability) zuerst auf einem Staging-Host durch.

2. System-Überblick für Betreiber

2.1 Die 25 Container, nach Belang gruppiert

Gruppe	Dienst	Rolle	Ausfall-Auswirkung
Edge	caddy	TLS-Terminierung, Routing, interne CA	alles extern
Identität	authentik-server, authentik-worker, authentik-db, authentik-redis	OIDC-IdP + Gruppen + Blueprints	Logins broken; Emergency-Token-Fallback
App-DB	postgres	rag.* -Schema	APIs liefern 5xx
Queue	redis	RQ-Queue, Pub/Sub	Ingestion stockt
Objekte	minio	Rohdateien (unveränderlich, versioniert)	Upload/Preview defekt
Vektoren	qdrant	Hybrid-Index + ACL-Filter	Retrieval defekt
LLM-Runtime	ollama, ollama-init, tei-reranker	Generierung + Embed + Rerank	Antworten defekt
Eigene Services	docling, ingestion-api, ingestion-worker, retrieval-api, duckdb-api	Das Produkt	Funktionen defekt
UI	openwebui, pipelines	Endnutzer-Chat	Benutzer sehen generischen OIDC-Fehler
Observability	langfuse, langfuse-db, prometheus, grafana, loki, promtail	Traces + Metriken + Logs	Debugging schwerer; Produkt läuft weiter
Automatisierung (Profil: automation)	n8n, watcher	Geplante Jobs + Ordner-Watch	Automatisierung stoppt

2.2 Host-seitig offene Oberfläche

Nur **80** und **443** werden von Docker auf dem Host veröffentlicht. Alles andere ist über Service-Namen im internen Bridge-Netzwerk `reineke` erreichbar.

Externe Pfade über Caddy:

URL	Ziel
/	Open WebUI
/auth/...	Authentik
/langfuse	Langfuse
/grafana	Grafana
/n8n/	n8n (Automation-Profil)

Interne Service-Ports siehe [docs/04_OPERATIONS.md](#) Anhang A.

2.3 Persistenz-Oberfläche

Ein Datenwurzelverzeichnis (`${DATA_ROOT}` , Standard `/var/lib/reineke`) enthält **ein Unterverzeichnis pro Dienst**. Das hält das Backup-Skript lesbar: ein Verzeichnis → eine Prozedur.

```
postgres/  authentik-db/  langfuse-db/  redis/  minio/  qdrant/
duckdb/    ollama/        tei/          docling/ loki/    grafana/
```

3. Host-Vorbereitung

3.1 Hardware

Ressource	Minimum	Referenz	Empfohlen
CPU / SoC	8 Kerne	Apple M4 Max (14 Kerne)	M4 Max oder Linux x86 + 24 GB GPU
RAM	32 GB	64 GB	64–128 GB
Disk (SSD)	250 GB	1 TB	1 TB + externes Backup
OS	macOS 14+ / Ubuntu 22.04+	macOS 15 auf M4 Max	—
Container- Runtime	Docker 24+ oder Colima 0.7+	Colima 0.7	—

Auf macOS Colima mit großzügiger Ressourcen-Zuweisung:

```
colima start --cpu 10 --memory 48 --disk 200
```

Mindestens 16 GB RAM für macOS selbst lassen. Docker Desktop funktioniert ebenfalls; mit demselben Budget.

3.2 DNS und TLS

- Internen DNS-Eintrag `rag.<firma>.local` → Host-IP setzen.
- TLS verwendet standardmäßig Caddys interne CA. Das CA-Zertifikat mit `scripts/export-ca.sh > ca.crt` exportieren und an Clients verteilen, damit keine Browser-Warnung erscheint.
- Für Let's Encrypt per DNS-01 das Caddyfile beim Build-Handover mit dem deployment-agent anpassen.

3.3 Ports

Nur **80** und **443** eingehend am Host. *Alle anderen Docker-Ports müssen auf der Bridge bleiben.* Hat der Host eine Firewall, alles außer SSH ablehnen.

3.4 Verzeichnisse

```
mkdir -p /var/lib/reineke /etc/reineke
sudo chown "$USER" /var/lib/reineke /etc/reineke
```

3.5 Geheimnisse

`scripts/bootstrap.sh` erzeugt fehlende Secrets und schreibt sie in `.env`. Vor dem ersten Lauf sollten Sie in einen Passwort-Manager ablegen:

- `AUTHENTIK_BOOTSTRAP_PASSWORD` (erstes Admin-Passwort, muss beim ersten Login geändert werden).
- `BACKUP_GPG_PASSPHRASE_FILE` (optional; für verschlüsselte Backups).
- `ALERT_WEBHOOK_URL` (optional).

Alles andere (`POSTGRES_PASSWORD` , `QDRANT_API_KEY` , `MINIO_ROOT_PASSWORD` , `INTERNAL_SERVICE_TOKEN` , `LANGFUSE_*` , `AUTHENTIK_SECRET_KEY`) kann automatisch erzeugt werden. Nach der Erzeugung **die gesamte** `.env` als Anhang im Passwort-Manager sichern.

4. Installation (online)

Sobald `services/**` vom Agenten-Build erzeugt wurde, erfolgt die Erstinstallation wie folgt:

```
# 1. Release klonen oder entpacken
cd /opt && tar xf reineke-rag-v1.0.0.tar.gz && cd reineke-rag

# 2. .env + owner-inputs.yaml erzeugen
bash scripts/bootstrap.sh

# 3. Container-Images ziehen (online)
make pull

# 4. LLM + Embedder + Reranker-Modelle ziehen (~80 GB bei schwerster Stufe)
make pull-models # oder: PULL_HEAVY=false make pull-models

# 5. Core-Stack starten
make up

# 6. Auf Healthchecks warten (2-5 min beim ersten Start)
make wait-healthy

# 7. Authentik-Bootstrap abschließen
open https://$PRIMARY_DOMAIN/auth/

# 8. Ordner + ACLs aus config/owner-inputs.yaml seeden
rag-admin folders sync config/owner-inputs.yaml

# 9. Retrieval-Smoke-Test
rag-admin query "Ping" # erwartet eine Ablehnung; beweist Auth + Retrie
```

Der Smoke-Test verweigert absichtlich — eine „keine Information gefunden“-Antwort ohne Leckage zeigt, dass (a) OIDC funktioniert, (b) Qdrant für einen unbekannten Begriff null Punkte zurückgab, (c) der Refusal-Stil korrekt greift.

4.1 Compose-Profile

Profil	Zweck	Aufruf
(Standard)	Core-19-Dienste	<code>make up</code>
<code>minimal</code>	Core ohne Langfuse/Loki/Grafana	<code>PROFILES=minimal make up</code>
<code>automation</code>	Core + n8n + watcher	<code>make up-automation</code>
<code>init</code>	Einmalig: zieht Ollama-Modelle und beendet sich	<code>make pull-models</code>

5. Installation (air-gapped)

Der Stack ist darauf ausgelegt, ohne ausgehenden Netzzugang zu laufen. Eine online-Helfer-Maschine baut ein ca. 100 GB großes Paket:

```
# Auf der Online-Helfer-Maschine
bash scripts/pack-offline.sh
# Erzeugt reineke-rag-offline-<date>.tar.gz mit:
# - docker save jedes Images (gepinnte Tags)
# - Ollama-Modell-Gewichte (bge-m3, gemma2:9b, qwen2.5:32b, llama3.3:70b)
# - TEI-Reranker-Gewichte
# - Docling-OCR-Modelle
# - Python-Wheels für die eigenen Services
```

Auf dem Zielsystem:

```
bash scripts/load-offline.sh reineke-rag-offline-<date>.tar.gz
make up
make wait-healthy
```

Jeder Laufzeit-Pfad (Retrieval, Ingestion, Auth) muss mit gezogenem LAN-Kabel funktionieren — das wird in Phase 9 (A9.3) getestet. Tauchen im Log ausgehende Verbindungen auf, ist das ein Bug.

6. Tagesbetrieb — das `rag-admin -CLI`

`rag-admin` ist ein schlanker Python-Wrapper, der sich als Admin authentifiziert und die drei eigenen APIs anspricht. Häufigste Befehle:

```

# Status + Gesundheit
rag-admin status                                # Container-Status, Queue-Tiefe, Dokument-Anza
rag-admin jobs list --state failed

# Benutzer
rag-admin users list
rag-admin users add alice@firma.de --groups engineering,qms

# Ordner + ACLs
rag-admin folders list
rag-admin folders create /qms/normen --groups admin,qms,engineering
rag-admin folders set      /qms/normen admin,qms,engineering,auditor
rag-admin folders move     /qms/normen /qms/standards          # schreibt Dokumentzeilen
rag-admin folders delete   /qms/normen --wait                  # verweigert, wenn noch D

# Dokumente
rag-admin docs list --folder /qms/normen
rag-admin docs upload ./drop/*.pdf --folder /qms/normen
rag-admin docs ingest-dir drop/ --base-folder /                # rekursiv
rag-admin docs reindex <doc_id>
rag-admin docs reindex --folder /qms/normen
rag-admin docs reindex --all --confirm                          # Rundumschlag
rag-admin docs delete <doc_id>      # soft; 30 Tage Retention
rag-admin docs delete <doc_id> --hard # erfordert ADMIN_CONFIRM=yes

# Anfragen (zum Testen)
rag-admin query "Welche Norm gilt für Typ-B-Schränke?"

# Modelle
rag-admin models list

# Backups
rag-admin backup run
rag-admin restore plan <backup-dir>      # Trockenlauf
rag-admin restore apply <backup-dir>

# Sessions + Audit
rag-admin sessions revoke-all
rag-admin audit export --from 2026-01-01 --to 2026-03-31 --format csv > q1.csv

# Alerts
rag-admin alerts silence <rule> --until 2026-05-01

```

Autoritative Liste in [docs/04_OPERATIONS.md §2.3](#). Jeder Befehl trifft eine HTTP-API; alles, was per CLI geht, geht auch per direktem Aufruf von `ingestion-api` oder `retrieval-api` mit einem Admin-JWT.

7. Benutzer- und Gruppenverwaltung

Benutzer und Gruppen werden in **Authentik** gepflegt. Die App-DB spiegelt das Minimum (`rag.users.id`, E-Mail, Gruppen) für Audit-Joins; der Spiegel aktualisiert sich bei jeder JWT-Validierung.

7.1 Benutzer anlegen

1. `https://$PRIMARY_DOMAIN/auth/` → Directory → Users → Create.
2. Name + E-Mail ausfüllen. Passwort leer lassen, um eine Einladungs-E-Mail zu senden (falls SMTP konfiguriert); sonst temporäres Passwort manuell setzen und out-of-band kommunizieren.
3. Directory → Groups → Gruppe(n) auswählen → Benutzer als Mitglied hinzufügen.
4. Beim nächsten Login landet der Benutzer unter `/` in der Chat-UI mit den richtigen Rechten.

7.2 Gruppe anlegen

1. Directory → Groups → Create. Den Namen wählen, der später in Ordner-ACLs verwendet wird (z. B. `auditor`).
2. `rag-admin folders set /<path> <groups>` für die entsprechenden Ordner-Rechte.
3. Warten, bis `rag-admin jobs list --kind reac1` abfließt (≤ 30 s pro 1 000 Chunks).

7.3 Offboarding

1. In Authentik den Benutzer auf Inactive setzen.
2. Access-Tokens (15 min) und Refresh-Tokens (24 h) laufen natürlich ab; effektive Sperrung in der Praxis < 1 min, weil der nächste Token-Refresh scheitert.
3. Chats werden laut Policy aufbewahrt; Audit-Einträge gemäß Retention-Regeln. Anonymisierungspfad siehe Entwurf zur Datenschutznotiz.

7.4 Passwort-Reset

- Per Authentik-Self-Service-Flow, falls SMTP konfiguriert ist.
- Sonst: Directory → Users → Passwort zurücksetzen (temporär) → out-of-band ausliefern.

7.5 Notfall: Authentik offline

`ADMIN_BACKUP_TOKEN` in `.env` ist ein langlebiges, ausschließlich für den Notfall vorgesehenes JWT, das Admin-Zugang zu `rag-admin` und den APIs gewährt. Alle 90 Tage rotieren. Jeder Einsatz wird deutlich im `audit_log` protokolliert. Niemals für den Normalbetrieb verwenden.

8. Ordner und ACLs

Der Ordnerbaum ist **logisch** (eine Datenbanktabelle), kein Dateisystem. `rag.folders` ist die Source of Truth; jedes Dokument trägt `folder_path` als Fremdschlüssel. `acl_groups TEXT[]` der Ordnerzeile wird in jedes Chunk-Payload (`acl_groups` als indiziertes Feld) und in `rag.documents.folder_path` kopiert.

8.1 Ordner pflegen

```
rag-admin folders create /qms/normen --groups admin,qms,engineering --description "QMS:
rag-admin folders set      /qms/normen admin,qms,engineering,auditor
rag-admin folders move    /qms/normen /qms/standards      # schreibt Zeilen um; kein Re-E
rag-admin folders delete  /qms/normen --wait              # verweigert, wenn noch Dokumen
```

8.2 ACL-Änderungs-Propagation

- Qdrant-**Payload-Rewrite**, kein Re-Embedding. Günstig — Qdrant aktualisiert das indizierte `acl_groups` -Feld in place.
- Ein `reacl` -Job läuft pro betroffenem Dokument; Fortschritt in `rag-admin jobs list`.
- DuckDB-Views (`views.v_<table>_<group_hash>`) werden lazy beim ersten Zugriff pro Gruppenset regeneriert.

8.3 Lesen vs. Schreiben (v1-Einschränkung)

v1 fasst Lese- und Schreibrechte unter „Gruppen-Mitgliedschaft“ zusammen. Wer eine Folder liest, kann dort auch hochladen (Admins ohnehin). v1.1 wird `acl_read` / `acl_write` trennen — als bekannte Einschränkung notiert.

8.4 Standard-ACL

Neue Dokumente ohne expliziten Ordner fallen auf `DEFAULT_FOLDER_ACL` (typisch `admin`). Die ingestion-api lehnt Uploads in nicht existierende Ordner ab — ein Schutz vor versehentlich öffentlichen Dateien.

9. Dokument-Ingestion und Lebenszyklus

9.1 Zustände

```
queued → parsing → embedding → indexed      (Normalfall)
                                   → failed      (bis zu 3× retryable)
                                   → superseded   (soft-gelöscht oder ersetzt)
```

Zustandsübergänge werden atomar in `rag.documents.status` geschrieben; hängt `parsing` oder `embedding` > 10 min, ist meist ein Container-Absturz schuld — `docker compose logs ingestion-worker` prüfen.

9.2 Unterstützte Formate

- **Text-PDF** — schneller Pfad, Struktur bleibt erhalten.
- **Gescanntes PDF** — OCR per EasyOCR (Standard) oder Tesseract (`OCR_LANG=deu+eng`). Langsamer, qualitativ OCR-abhängig. Ziel > 90 % Text-Wiedergewinnung am Fixture-Set.
- **DOCX** — Abschnitte, Listen, Tabellen.
- **XLSX** — jedes Sheet als Chunk eingebettet *und* als typisierte Tabelle in DuckDB geladen.
- Optional (nach Phase 5 formatweise aktivierbar): **PPTX**, **HTML**, **MD**.

9.3 Bulk-Ingestion

```
mkdir -p drop/qms/normen
cp /Volumes/Share/QMS/*.pdf drop/qms/normen/

rag-admin docs ingest-dir drop/ --base-folder /
# oder Trockenlauf:
rag-admin docs ingest-dir drop/ --base-folder / --dry-run
```

Fortschritt auf dem **Ingestion**-Grafana-Dashboard:

- Queue-Tiefe
- Durchsatz MB/s
- Parse-Fehler pro MIME-Typ
- Top-Problemdokumente nach Fehlercode

9.4 Deduplizierung

SHA-256 des Datei-Inhalts ist der Dedup-Schlüssel. Dieselbe Datei in denselben Ordner → HTTP 409 mit bestehender `doc_id`; kein Reparse, kein Re-Embed. Dieselbe Datei in einen anderen Ordner ist ein neues Dokument (andere ACL).

9.5 Versionierung

Das Hochladen einer gleichnamigen Datei mit abweichendem SHA-256 erzeugt eine neue `doc_id` und markiert die vorherige als `superseded`. Die alten Chunks werden aus dem Index entfernt; die alten Bytes bleiben 30 Tage in MinIO unter `raw/{old_id}/...`.

9.6 Reindexing

Nötig, wenn:

- Sich die Chunking-Konfiguration ändert (`CHUNK_MAX_TOKENS`, `PRESERVE_TABLES` usw.).
- Ein Parser-Upgrade Qualitätszuwachs erwarten lässt (per `scripts/eval.py` messen).
- Eine Quelldatei in-place mutiert wurde (selten; besser Versionierung nutzen).

```
rag-admin docs reindex <doc_id>
rag-admin docs reindex --folder /qms/normen
rag-admin docs reindex --all --confirm          # Minuten bis Stunden
```

9.7 Löschen

- **Soft-Delete** (Standard): setzt `superseded`, entfernt aus dem Index; Bytes bleiben 30 Tage in MinIO.
- **Hard-Delete**: Flag `--hard`, erfordert `ADMIN_CONFIRM=yes`. Löscht Bytes + alle Zeilen + alle Qdrant-Punkte. Irreversibel.

9.8 Fehlgeschlagene Ingestions

- Passwortgeschützte PDFs: Fehlertext weist darauf hin; Eigentümer muss eine entsperrte Kopie liefern, oder Admin aktiviert `PDF_UNLOCK_ATTEMPTS=true` (nur Leerpasswort-Versuch).
- Docling-Parse-Fehler: `PARSER_FALLBACK=pymupdf` führt das Dokument einmalig über einen leichten Text-Pfad aus. Qualität sinkt; bitte dokumentieren.
- Zu große Dateien: `INGEST_MAX_BYTES` erhöhen, aber Worker-RAM beobachten.

10. Retrieval-Verhalten und Feintuning

10.1 Die vier Frageklassen

Klasse	Beispiel	Pfad	LLM-Stufe
Lookup	„Welche Norm gilt für Typ-B-Schränke?“	Hybrid + Rerank	Gemma 2 9B
Extraction	„Liste alle Lieferfristen aus Angebot-2024-09.pdf.“	Hybrid + Rerank, langer Kontext	Qwen 2.5 32B
Table-math	„Welches Projekt hatte 2024 die höchste Marge?“	Paralleles SQL + Hybrid für Kontext	Qwen 2.5 32B
Synthesis	„Fasse unsere Position zu Thema X über alle QMS-Dokumente zusammen.“	Hybrid + Rerank Top 20, Map-Reduce	Llama 3.3 70B

Der Klassifizierer ist ein einmaliger Gemma-9B-Aufruf mit versioniertem Prompt. Langfuse protokolliert Klassifikation + Confidence bei jeder Anfrage — Drift über das Quality-Dashboard beobachten.

10.2 Retrieval-Algorithmus (Kurzfassung)

```

classify → optionales Rewriting (HyDE/Paraphrase x2)
→ embed(query) dense + sparse
→ Qdrant-prefetch: dense Top 50 + sparse Top 50, beides mit ACL-Filter
→ RRF-Fusion → Top 50
→ TEI-Rerank → Top 12
→ (falls table-math) SQL-Zweig parallel: LLM → duckdb-api → Zeilen
→ Prompt bauen (DE/EN/zweisprachig)
→ Ollama stream mit gewählter Stufe
→ SSE-Tokens + citations-Event
→ Audit + Langfuse-Trace

```

Jeder Qdrant-Aufruf kopiert denselben ACL-Filter — **es gibt genau einen Such-Codepfad**. Harte Regel ([retrieval-agent-Briefing](#)).

10.3 Stellschrauben

Alles in `.env` :

Schraube	Default	Wirkung	Reindex nötig?
TOP_K_DENSE	50	Kandidaten aus Dense-Suche	Nein
TOP_K_SPARSE	50	Kandidaten aus Sparse-Suche	Nein
TOP_K_RERANK	12	Kandidaten nach Rerank → Prompt	Nein
HYBRID_FUSION	rrf	oder dbsf (Qdrant Distribution-Based Score Fusion)	Nein
QUERY_REWRITE	true	Erzeugt 2 Paraphrasen	Nein
CHUNK_MAX_TOKENS	512	Obergrenze des Chunkers	Ja
CHUNK_MIN_TOKENS	128	Untergrenze des Chunkers	Ja
PRESERVE_TABLES	true	Tabellen bleiben intakt	Ja
CITATION_STYLE	brackets	[1] inline oder Fußnoten	Nein
REFUSAL_ON_EMPTY	true	Nie fabrizieren; ablehnen ohne Stützung	Nein

10.4 Prompt-Templates

Dateien auf der Festplatte unter `config/retrieval/prompts/{de,en,bilingual}.md`. Versioniert wie Code. Ein Laufzeitwechsel erfordert Bearbeitung der Datei und Neustart von `retrieval-api` — es gibt in v1 keinen Hot-Admin-UI für Prompts.

Jede Prompt-Änderung muss auf dem 50-Fragen-Gold-Set (`scripts/eval.py`) Parität oder Zuwachs zeigen, bevor sie ausgerollt wird. Baselines liegen unter `docs/eval/baseline-YYYY-MM-DD.md`.

10.5 Refusal-Stil

Enthält das rerankte Top-K keinen Chunk, der die Antwort stützt:

- DE: „Ich habe dazu in den zugänglichen Dokumenten keine Information gefunden.“
- EN: „I didn't find information on that in the documents you can access.“

Keine Teil-Fabrikation. Kein „vielleicht so, vielleicht so“. Die *Ablehnung* selbst ist Teil des Produkts.

11. Modell-Management

11.1 Modell-Katalog

Zweck	Default	Größe	RAM	Env-Variable
Schnelles LLM	<code>gemma2:9b-instruct-q5_K_M</code>	~7,5 GB	~7,5 GB	<code>LLM_FAST</code>
Reasoning-LLM	<code>qwen2.5:32b-instruct-q4_K_M</code>	~20 GB	~20 GB	<code>LLM_REASONING</code>
Heavy-LLM	<code>llama3.3:70b-instruct-q4_K_M</code>	~40 GB	~40 GB	<code>LLM_HEAVY</code>
Embedder	<code>bge-m3</code>	~1 GB	~1 GB	<code>EMBED_MODEL</code>
Reranker	<code>BAAI/bge-reranker-v2-m3</code>	~0,6 GB	~0,6 GB	<code>RERANK_MODEL</code> (TEI)

Ollama lädt lazy und räumt auf Basis von `OLLAMA_MAX_LOADED_MODELS` (Default 2 auf 64 GB).
`OLLAMA_KEEP_ALIVE=10m` bedeutet: inaktive Modelle werden nach 10 Minuten entladen.

11.2 Router-Config

`config/retrieval/models.yaml` (Eigentum `llm-agent`) ist der einzige Ort, um Klasse → Modell zu ändern:

```
classes:
  lookup:      { model: gemma2:9b-instruct-q5_K_M,  max_tokens: 400 }
  extraction:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 1200 }
  table-math:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 800 }
  synthesis:   { model: llama3.3:70b-instruct-q4_K_M, max_tokens: 1600 }
  embedding:   { model: bge-m3, dimensions: 1024 }
  reranker:    { model: BAAI/bge-reranker-v2-m3, server: tei }
```

Neustart: `docker compose restart retrieval-api`. Kein Reindex nötig.

11.3 Modelle wechseln

```
# Neue Variante ziehen
docker compose exec ollama ollama pull qwen2.5:32b-instruct-q5_K_M

# models.yaml editieren; Neustart
docker compose restart retrieval-api

# Gegen das Gold-Set evaluieren
python scripts/eval.py --gold config/eval/gold-queries.yaml

# Die bessere Variante (Recall@10 / Zitiergenauigkeit) behalten
```

Altes Modell behalten, bis die Neue validiert ist; es ist nur Disk.

11.4 Fallback für kleinere Hosts

`LLM_PROFILE=compact` verwirft die Heavy-Stufe und leitet `synthesis` auf `reasoning`. Auf 48-GB-Hosts akzeptabel. Dokumentieren; Größen-Matrix in [ADR-004](#) zeigt, was wohin passt.

11.5 Embedder-Wechsel = Breaking Change

Ändern von `EMBED_MODEL` löst aus:

1. Neue Qdrant-Collection (anderes Vektor-Format → frische Collection).
2. Voll-Reindex aller Dokumente.
3. MAJOR-Stack-Version-Bump.

Wartungsfenster einplanen. Die neue Collection wird blue/green angelegt; Anfragen gehen weiter gegen die alte, bis der Reindex abgeschlossen ist — dann flippt ein Cutover-Flag.

12. Observability

Drei Signale, bewusst getrennt:

12.1 Langfuse — LLM-Traces

- URL: `https://$PRIMARY_DOMAIN/langfuse/`.
- Ein Eltern-Trace pro Anfrage mit Kind-Spans: `classify`, `rewrite`, `dense_search`, `sparse_search`, `rerank`, `sql_plan`, `sql_exec`, `generate`.
- Jeder Span enthält: Modell, Eingaben (gekürzt), Ausgaben, Latenz, Token-Zählungen, Kosten (ungenutzt, aber emittiert).
- Filter für die Fehlersuche:
 - `latency > 10s` — langsame Anfragen finden.
 - `retrieval.rerank_score_top < 0.4` — Retrievals geringer Confidence.
 - Nach Benutzer-E-Mail — Beschwerde-Triage.

Replay aus dem Langfuse-UI mit anderem Modell für A/B-Vergleiche.

12.2 Prometheus + Grafana — Metriken

Vier provisionierte Dashboards:

Dashboard	Kern-Panels
Overview	QPS, p50/p95-Latenz pro Klasse, Fehlerrate, aktives Modell, <code>rag_build_info{version}</code>
Ingestion	Queue-Tiefe, Job-Zustands-Pie, Durchsatz MB/s, Parse-Fehler pro MIME
Infra	Container CPU/RAM, freie Disk, Netz, Docker-Log-Volumen
Quality	(aus Langfuse-Exporten gespeist) Rerank-Uplift, Refusal-Rate, Top „kein Zitat“-Queries

Wichtige Custom-Metriken:

- `rag_query_total{class}` — Counter
- `rag_query_latency_seconds{phase,class}` — Histogram
- `rag_retrieval_hits{source=dense|sparse|rerank}` — Counter
- `rag_ingestion_jobs_total{state}` — Gauge
- `rag_doc_count`, `rag_chunk_count` — aus Postgres, alle 60 s aktualisiert

12.3 Loki + Promtail – Logs

- Gesamter Container-Stdout fließt in Loki.
- Retention: 14 Tage INFO, 90 Tage WARN+.
- Einsehen in Grafana Explore → Loki-Datasource → `{container="retrieval-api"}`.

12.4 Alerts

Kanäle: `$_{DATA_ROOT}/alerts.log` stets; optionaler Webhook über `ALERT_WEBHOOK_URL` (Teams, Slack, Mattermost).

Standardregeln:

Regel	Schwelle	Schweregrad
<code>rag_disk_free_pct < 10</code>	anhaltend	kritisch
Beliebiger Container unhealthy > 5 min	—	kritisch
<code>rag_ingestion_queue_depth > 200</code>	—	Warnung
<code>rag_ingestion_queue_depth > 500</code>	—	kritisch
Kein Backup seit 26 h	—	kritisch
p95-Latenz <code>lookup</code> > 8 s, 10 min am Stück	—	Warnung

Stummschalten: `rag-admin alerts silence <rule> --until 2026-05-01`.

13. Backup und Restore

13.1 Was gesichert wird

Quelle	Verfahren	Typische Größe
postgres (rag-DB)	<code>pg_dump -Fc</code>	« 1 GB
authentik-db	<code>pg_dump -Fc</code>	< 100 MB
langfuse-db	<code>pg_dump -Fc</code>	1 GB/Monat Wachstum
minio	<code>mc mirror</code>	~ Rohkorpus + 5 %
qdrant	Snapshot-API → Tarball	skaliert mit Chunks
duckdb	Dateikopie	zehn MB
redis	AOF-Dateikopie	< 10 MB

Was **nicht** gesichert wird: Ollama-Modell-Gewichte (Re-Pull von Quelle), TEI-/Docling-Caches (regenerieren sich), Loki (ephemer), Prometheus-TSDB (ephemer), Grafana-State (aus Config provisioniert).

13.2 Zeitplan

- Nächtlich um **02:15 lokal** per launchd (macOS) oder systemd-Timer (Linux).
- Retention-GFS: **7 tägliche / 4 wöchentliche / 12 monatliche**.
- Ausgabe: `${BACKUP_ROOT}/YYYY-MM-DD/`.
- Optionale GPG-Verschlüsselung: `BACKUP_GPG_PASSPHRASE_FILE` setzen.

13.3 Manueller Lauf

```
rag-admin backup run
ls -lh "$BACKUP_ROOT/$(date +%F)/"
```

13.4 Restore-Probe (Pflicht)

```
make down
sudo mv /var/lib/reineke /var/lib/reineke.old
sudo mkdir /var/lib/reineke && sudo chown "$USER" /var/lib/reineke

rag-admin restore plan "$BACKUP_ROOT/2026-04-22/"
rag-admin restore apply "$BACKUP_ROOT/2026-04-22/"
make up
make wait-healthy
rag-admin query "Welche Norm gilt für Typ-B-Schränke?" # Sanity-Check
```

Mindestens einmal pro Major-Upgrade proben. Ein nie zurückgespieltes Backup ist eine Hoffnung, kein Backup.

13.5 Verschlüsselung im Ruhezustand

- macOS: FileVault auf der Datenplatte.
- Linux: LUKS auf `${DATA_ROOT}` oder ZFS-Native-Encryption.
- Backup-Medium: GPG-symmetrisch mit 25-stelliger Zufalls-Passphrase; Passphrase im Passwort-Manager; Pfad in `BACKUP_GPG_PASSPHRASE_FILE`.

14. Security-Operations

14.1 Vertrauensmodell (Kurzfassung)

- Nur Caddy bindet Host-Ports.
- Alle Service-zu-Service-JWT-Validierungen gehen durch `services/common/auth.py` (Shared-Library) gegen den Authentik-JWKS-Endpunkt. RS256, 2048 Bit.
- Interne API-Aufrufe nutzen `INTERNAL_SERVICE_TOKEN` (einzelnes Shared Secret, per `scripts/rotate-secrets.sh` rotiert).
- Qdrant: API-Key, nur bei den APIs hinterlegt.
- MinIO: IAM-Keys; Browser erhalten Pre-signed URLs.
- Ollama: keine Auth; verlässt sich auf Netzisolierung. Niemals an den Host binden.

14.2 Credential-Rotation

```
bash scripts/rotate-secrets.sh
```

Rotiert Postgres-Passwörter, Qdrant-API-Key, MinIO-Root-Keys, internen Service-Token. Rollierende Service-Neustarts. Dauer < 1 min.

Authentik-Admin-Konto: über das UI rotieren; neue Recovery-Secret in den Passwort-Manager.

14.3 TLS

Caddy rotiert die interne CA alle 90 Tage. Beim Rotieren die neue CA an die Clients verteilen:

```
scripts/export-ca.sh > ca.crt  
# Verteilung per MDM oder manuell
```

Für Let's Encrypt per DNS-01 liegt das im Caddyfile und läuft automatisch.

14.4 Incident-Response

Szenario	Maßnahme
Verdacht auf Credential-Leak	<code>rag-admin sessions revoke-all</code> ; <code>bash scripts/rotate-secrets.sh</code> ; <code>audit_log</code> der letzten 30 Tage auf ungewöhnliche Anfragen prüfen; kontrollieren, dass <code>ADMIN_BACKUP_TOKEN</code> nicht unprotokolliert benutzt wurde
Host-Kompromittierung vermutet	<code>make down</code> ; Disk imagen; vom letzten sauberen Backup auf frischen Host zurückspielen; alles rotieren, bevor Benutzer wieder zugelassen werden
Benutzer meldet falsche/leakende Antwort (PII, andere Gruppe)	<code>audit_log</code> -Zeile und Quell-Qdrant-Punkt sichern; ACL des Quelldokuments prüfen; gezielte Löschung oder Ordner-Verschiebung erwägen

14.5 Audit-Export (GDPR)

```
rag-admin audit export --from 2026-01-01 --to 2026-03-31 --format csv > q1.csv
```

Felder definiert in [docs/02_ARCHITECTURE.md §4](#). Für Betroffenen Auskunft zu eigenen Datensätzen: nach `user_id` filtern.

15. Upgrades

15.1 Patch/Minor (v1.0.0 → v1.0.1)

```
git fetch && git checkout v1.0.1
make pull
make up
make wait-healthy
```

Erst das Changelog auf **migration**- oder **reindex**-Hinweise lesen.

15.2 Major (v1.x.y → v2.0.0)

```
make backup
git checkout v2.0.0
make pull
rag-admin migrate preflight      # listet erforderliche Änderungen
rag-admin migrate apply         # queued Reindex-Jobs; Grafana-Ingestion beobachten
```

30–120 min Fenster je nach Korpus-Größe einplanen. Embedding-Wechsel sind am teuersten — jeder Chunk wird neu embedded.

15.3 Ollama-Modell-Upgrades

```
docker compose exec ollama ollama pull qwen2.5:32b-instruct-q5_K_M
# config/retrieval/models.yaml editieren; dann:
docker compose restart retrieval-api
python scripts/eval.py          # Recall + Zitiergenauigkeit vergleichen
```

Nur den Sieger behalten.

16. Performance-Tuning

16.1 Latenz-Budgets

Klasse	Ziel p95 bis zum letzten Token
lookup	≤ 5 s
extraction	≤ 15 s
table-math	≤ 20 s
synthesis	≤ 60 s

Bei Überschreitung den schlimmsten Span im Langfuse-Trace suchen.

16.2 Übliche Tuning-Hebel

Symptom	Maßnahme	Kosten
<code>lookup -p95</code> driftet auf 8 s+	<code>CHUNK_MAX_TOKENS</code> auf 300 senken, reindexen; Modell-Eviction prüfen	Reindex-Zeit
Geringer Recall auf Fließtext	<code>CHUNK_MAX_TOKENS</code> auf 800 erhöhen, reindexen	Reindex-Zeit
Rerank zu langsam	<code>TOP_K_DENSE</code> + <code>TOP_K_SPARSE</code> auf 30 senken	Leichter Recall-Verlust
Synthesis zu langsam auf M4 Max	<code>LLM_PROFILE=compact</code> (70B raus)	Qualitätseinbuße bei Synthese
Memory-Thrash	Prüfen <code>OLLAMA_MAX_LOADED_MODELS=2</code> , <code>OLLAMA_NUM_PARALLEL=1</code>	—
Qdrant-RAM-Druck	<code>quantization.scalar.type=int8</code> aktiv prüfen; <code>on_disk_payload=true</code> lassen	Vernachlässigbare Qualitätseinbuße

16.3 Disk-Hygiene

- `docker system df`; wöchentlich `docker system prune -f --filter until=168h`.
- `ollama list` zeigt gecachte Modelle; `ollama rm <old>` gibt Platz frei.
- Prometheus-TSDB-Retention ist 15 Tage; nur erhöhen, wenn Platz vorhanden.

17. Fehlersuche

17.1 Allgemeine Methode

- `docker compose ps` — was ist unhealthy?
- `docker compose logs --since 30m <service>` — jüngste Fehler.
- `rag-admin status` — Anwendungs-Zustand (Queue-Tiefe, Dokumentanzahl, letzte Anfrage).
- Langfuse oder Grafana für symptomspezifische Dashboards.

17.2 Symptom → Ursachen-Tabelle

Symptom	Wahrscheinliche Ursache	Nächster Schritt	
Alles antwortet „keine Information gefunden“	Fehlender <code>groups</code> - Claim im JWT	Authentik-OIDC-Property-Mappings; <code>retrieval-api</code> -Log zeigt <code>groups=[]</code>	
Ingestion hängt bei <code>parsing</code>	Docling-Exception (oft verschlüsselte PDF)	<code>docker compose logs ingestion-worker</code> ; nach Fix retry	
Ingestion hängt bei <code>embedding</code>	Ollama lädt kalt oder OOM	<code>ollama ps</code> ; RAM prüfen; <code>OLLAMA_MAX_LOADED_MODELS</code> senken	
„Alle Anfragen langsam“	Zwei Heavy-Modelle gleichzeitig geladen	<code>ollama ps</code> ; Parallelität senken; <code>OLLAMA_KEEP_ALIVE</code> reduzieren	
Antwort zitiert den falschen Chunk	Retrieval-Präzisionsproblem	Langfuse-Trace → Rerank-Scores; fehlt der Gold-Chunk, re-chunk / k erhöhen; ist er da aber schlecht gereiht, Reranker / Sprach-Mismatch	
Antwort in falscher Sprache	Modell-Sprachpräferenz	„Answer in <lang>.“ in System-Prompt oder Nutzer-Toggle	
Zitat-Preview 404	MinIO-Pre-signed-URL abgelaufen (> 1 h)	Erneut klicken; bei anhaltendem Problem MinIO down	
ACL-Leak (Benutzer sah fremd-Gruppen-Chunk)	Bug ; ACL-Filter wurde auf einem Pfad übergangen	Trace sichern, an retrieval-agent eskalieren; KRITISCH	
Kein Backup letzte Nacht	Backup-Skript gescheitert	<code>alerts.log</code> prüfen; macOS-Cron verpasst? <code>`launchctl list`</code>	<code>grep reineke`</code>
Qdrant-Collection-Korruption	Selten	Snapshot zurückspielen (<code>scripts/qdrant-snapshot.sh apply <snap></code>); Fallback ist Full-Reindex	

17.3 Eskalation

Drei Wiederholungen desselben Abnahmekriteriums während des Builds → der Koordinator hält an und eskaliert. In Produktion: drei Wiederholungen desselben Runbook-Fehlers → Ticket eröffnen, nicht endlos wiederholen.

18. Das System erweitern

Dokumentierte Erweiterungspunkte. Jeder ist eine begrenzte Änderung mit bekanntem Migrationsmuster.

18.1 Neuer MIME-Typ

1. Parser-Zweig in `services/docling/app.py` einfügen.

2. Fixtures unter `tests/fixtures/` ergänzen.
3. `SUPPORTED_MIME_TYPES` -Env in `ingestion-api` erweitern.
4. Keine Schema-Änderung.

18.2 Neues LLM

```
docker compose exec ollama ollama pull <new-model>
# config/retrieval/models.yaml editieren
docker compose restart retrieval-api
python scripts/eval.py          # messen
```

18.3 Neuer Embedder (breaking)

- Neue Qdrant-Collection (anderes Vektor-Format).
- Blue/Green-Reindex.
- MAJOR-Version-Bump.
- `EMBED_MODEL` -Env + `models.yaml.embedding.dimensions` aktualisieren.

18.4 Neues ACL-Prädikat (z. B. Vertraulichkeitsstufe)

1. Postgres-Migration: Spalte auf `rag.documents`.
2. Indiziertes Qdrant-Payload-Feld hinzufügen.
3. Filterklausel im *einzigsten* Retrieval-Codepfad erweitern.
4. Backfill-Skript aktualisiert bestehende Dokumente.
5. Optional: Sichtbarkeit im Admin-UI.

18.5 Neue Datenquelle (Wiki, Mail)

- Connector-Service schreiben, der DoclingDocument-ähnliches JSON erzeugt.
- An den bestehenden `ingestion-api` -Endpunkt mit `folder_path` + ACLs senden.
- Keine Retrieval-Änderung nötig.

18.6 Zwei-Host-Split (v1.1)

Wenn Korpus > 5 000 Dokumente UND Team > 30 aktive Nutzer:

- Worker-Host: Ollama + TEI + ingestion-worker (schwer in CPU/Metal).
- App-Host: alles andere.
- Docker-Overlay-Network oder WireGuard dazwischen.
- `scripts/split-to-two-hosts.sh` erzeugt die Override-Datei. In Staging proben.

19. Anhänge

19.1 Interne Port-Map

Dienst	Port	Hinweis
caddy	80, 443 (Host)	Nur Host-Bindung
authentik-server	9000	UI + OIDC
postgres	5432	rag -DB
redis	6379	Queue + Pub/Sub
minio	9000 (S3), 9001 (Konsole)	IAM-Credentials
qdrant	6333 (REST), 6334 (gRPC)	API-Key-Auth
ollama	11434	keine Auth
tei-reranker	8080	intern
docling	8001	intern
ingestion-api	8010	JWT
retrieval-api	8020	JWT
duckdb-api	8030	JWT
openwebui	8080	gereverst
pipelines	9099	intern
langfuse	3000	unter /langfuse gereverst
prometheus	9090	intern
grafana	3000	unter /grafana gereverst
loki	3100	intern

19.2 Kapazitätsplanung (Daumenregel)

Korpus	Qdrant-Disk	Qdrant-RAM (int8)	MinIO-Disk	Postgres
500 Dokumente (~50 k Chunks)	~0,5 GB	~0,2 GB	Roh × 1,05	~0,5 GB
2 000 Dokumente (~200 k Chunks)	~2 GB	~0,8 GB	Roh × 1,05	~1,5 GB
10 000 Dokumente (~1 M Chunks)	~10 GB	~4 GB	Roh × 1,05	~8 GB
50 000 Dokumente (~5 M Chunks)	~50 GB	~20 GB	Roh × 1,05	~40 GB

Ollama-Modell-Obergrenze: ~80 GB (alle Stufen + Embedder + Reranker). ≥ 20 % Disk-Reserve halten.

19.3 Nützliche Einzeiler

```
# Chunks zählen
docker compose exec postgres psql -U rag -c "select count(*) from rag.chunks;"

# Top 10 langsame Anfragen der letzten 24 h
docker compose exec postgres psql -U rag -c "
select substring(query,1,80), latency_ms
from rag.audit_log
where ts > now() - interval '24 hours'
order by latency_ms desc limit 10;"

# Qdrant-Sanity
curl -s -H "api-key: $QDRANT_API_KEY" http://localhost:6333/collections/chunks | jq .

# Aktive Sessions (letzte 5 min)
docker compose exec postgres psql -U rag -c "
select count(distinct user_id) from rag.audit_log
where ts > now() - interval '5 minutes';"

# Welche Modelle sind in Ollama geladen
docker compose exec ollama ollama ps
```

19.4 Stilllegung

```
make down
rag-admin backup run                # finales Backup
# ${BACKUP_ROOT} in Cold Storage archivieren; Prüfsumme verifizieren
# ${DATA_ROOT} entfernen
# Authentik-OIDC-Clients widerrufen
# DNS entfernen + TLS-Zertifikat-Verteilung bereinigen
```

19.5 Wann zum Build-Team eskalieren

- Jede Abnahmekriterium-Regression nach einem Upgrade (A1.1–A10.1 in [docs/05_IMPLEMENTATION_PLAN.md](#)).
- Bestätigter ACL-Leak (kritisch; erzwingt Reöffnung des retrieval-agent-Pfads).
- Docling liefert nach einem Upgrade deutlich schlechtere Parses — `scripts/eval.py` laufen lassen und Diff mitliefern.
- Vorschlag, einen neuen Container aufzunehmen oder eine ADR-Entscheidung zu ändern — erfordert ein neues ADR, das das alte ersetzt; nicht daran vorbeikonfigurieren.

Verwandte Dokumente in diesem Repository:

- [TECH_DESCRIPTION_DE.md](#) — konzeptioneller Überblick, ADR-Index
- [USER_HANDBOOK_DE.md](#) — Handbuch für Endnutzer
- [docs/04_OPERATIONS.md](#) — autoritative Betriebs-Referenz (dieses Handbuch ist der Arbeitsbegleiter dazu)
- [docs/02_ARCHITECTURE.md](#) — autoritative Architektur-Referenz
- [docs/adr/](#) — Entscheidungsbegründungen